Sardar Patel Institute of Technology - Department of Computer Engineering
Cryptography and Network Security Lab

Experiment 8: Evaluation of Classical and Post-Quantum Digital Signature Algorithms using Docker-Based Client-Server Communication

Lab Report

Name: Anusha Dalal UID: 2024300032 Date: 16 October 2026

Section	Submission item (Section 19)	Page content
1	Hand-drawn digital-signature diagram (Part A) with name, UID and date	Handwritten sheet
2	Table 1: execution environment + screenshots of docker ps, docker network inspect sig-net, ping test	Table 1, Screenshots 1-6
3	Screenshots of key generation, signing, transfer (incl. sender's HTTP log) and verification	Screenshots 7-12
4	Table 2 (sizes), Table 3 (tampering, original and modified text), Table 4 (timings)	Tables 2-4, Screenshots 13-19
5	Seven graphs (Section 15), Tables 5, 6 and 7	Graphs 1-7, Screenshot 20, Tables 5-7
6	Answers to all discussion questions	Handwritten pages
7	200-250 word conclusion	Handwritten page
8	Source code of common.py, keygen.py, sign.py, verify.py, tamper.py, bench.py, plot.py and the Dockerfile	Listings
9	results.csv files (sender, receiver) and UID	Tables

1. Hand-drawn digital-signature diagram (Part A)

Name: Anusha Dalal UID: 2024300032 Date: _____

Drawn with pen/pencil and ruler: Sender (Alice), insecure channel, Receiver (Bob); M, H( ), h = H(M); SK_A (dashed, never sent) and PK_A; S = Sign(SK_A, h), output (M || S); PK_A distribution; h' = H(M'), Verify(PK_A, h', S); ACCEPT / REJECT decision box; Mallory modifying M to M' -> REJECT.

[attach handwritten pages here]

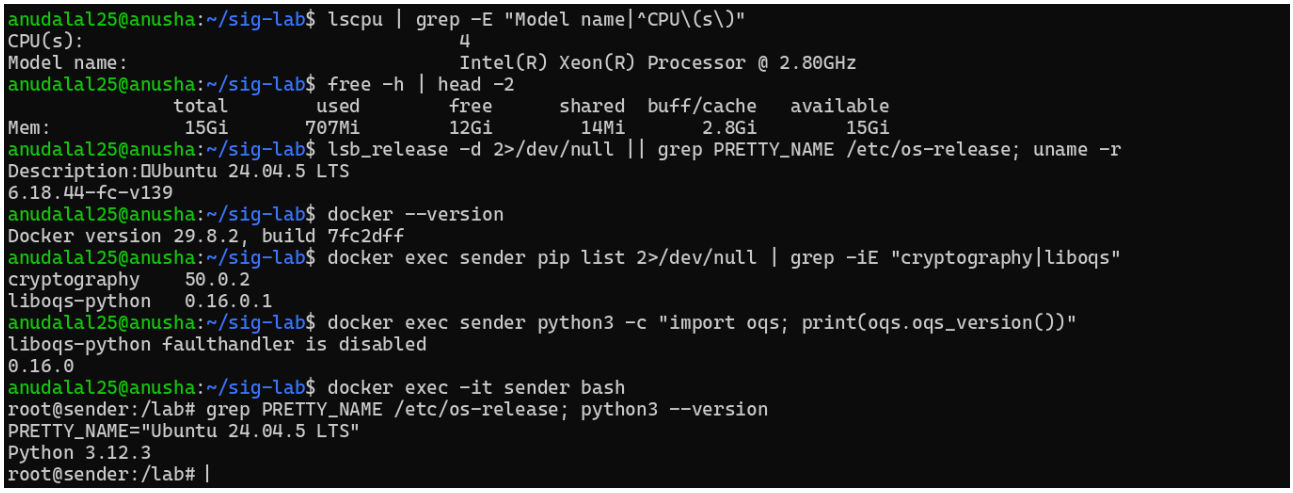
2. Execution environment and Docker setup (Part C)

Table 1: Execution environment

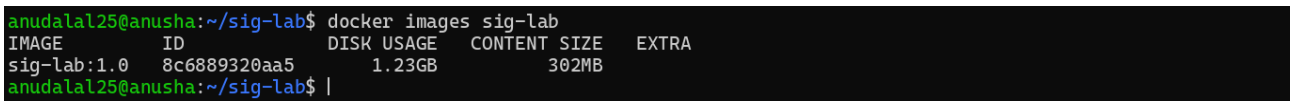
CPU model	Cores / threads	RAM	Host OS	Docker version	cryptography version	liboqs version
Intel(R) Xeon(R) Processor @ 2.80GHz	4 / 4	15 GiB	Ubuntu 24.04.5 LTS (kernel 6.18.44-fc-v139)	29.8.2	50.0.2	0.16.0 (liboqs-python 0.16.0.1)

Host OS: Ubuntu 24.04.5 LTS. Container OS (image sig-lab:1.0, built from the Step C3 Dockerfile, FROM ubuntu:24.04): Ubuntu 24.04.5 LTS, Python 3.12.3. Container IPs on sig-net: sender 172.18.0.2, receiver 172.18.0.3.

Screenshot 1: Step C8: execution environment (CPU, RAM, OS, Docker, cryptography and liboqs versions) for Table 1



Screenshot 2: Step C4: the image sig-lab:1.0 has been built and is listed by docker images



Screenshot 3: Step C5: user-defined bridge network sig-net created and listed (no containers attached yet)

```
anudalal25@anusha:~/sig-lab$ docker network create sig-net
144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6
anudalal25@anusha:~/sig-lab$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
f75c0c8e7cd5   bridge    bridge    local
482ed9f0038a   host      host      local
0da95fc264de   none      null      local
144a63bb1ae1   sig-net   bridge    local
anudalal25@anusha:~/sig-lab$ docker network inspect sig-net
[
    {
        "Name": "sig-net",
        "Id": "144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6",
        "Created": "2026-10-16T14:17:25.971919913Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
        "Containers": {},
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 3,
                        "DynamicIPsAvailable": 65533
                    }
                }
            }
        }
    }
]
anudalal25@anusha:~/sig-lab$
```

Screenshot 4: Step C6: sender and receiver containers started on sig-net with STUDENT_UID; docker ps shows both Up

```
anudalal25@anusha:~/sig-lab$ export STUDENT_UID=2024300032
anudalal25@anusha:~/sig-lab$ docker run -dit --name sender   --hostname sender   --network sig-net -e STUDENT_UID=20
24300032 -v ~/sig-lab/sender:/lab   sig-lab:1.0
26c3c6c0a3dfa5459e45d83f4b69c814c99f2997e96dd3e0abd26dce34c4216e
anudalal25@anusha:~/sig-lab$ docker run -dit --name receiver --hostname receiver --network sig-net -e STUDENT_UID=20
24300032 -v ~/sig-lab/receiver:/lab sig-lab:1.0
479e1892431dea47068dc938d27c4de070dcbfa620ec9c71485f6351f3a98440
anudalal25@anusha:~/sig-lab$ docker ps
CONTAINER ID   IMAGE         COMMAND            CREATED                  STATUS                  PORTS     NAMES
479e1892431d   sig-lab:1.0   "sleep infinity"   Less than a second ago   Up Less than a second             receiver
26c3c6c0a3df   sig-lab:1.0   "sleep infinity"   Less than a second ago   Up Less than a second             sender
anudalal25@anusha:~/sig-lab$
```

Screenshot 5: Step C7: docker network inspect sig-net now lists both containers with their IP addresses

```
anudalal25@anusha:~/sig-lab$ docker network inspect sig-net
[
    {
        "Name": "sig-net",
        "Id": "144a63bb1ae16f09cc5f469a969c01a79c54da674c4e2c4f09a87715288056b6",
        "Created": "2026-10-16T14:17:25.971919913Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": {},
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {},
        "Labels": {},
        "Containers": {
            "26c3c6c0a3dfa5459e45d83f4b69c814c99f2997e96dd3e0abd26dce34c4216e": {
                "Name": "sender",
                "EndpointID": "e2670e9c06aa1e248d73c5515dadecd49d22bc802fc395dd0d035a39ba6ca7e6",
                "MacAddress": "be:b3:ac:3b:b5:fb",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            },
            "479e1892431dea47068dc938d27c4de070dcbfa620ec9c71485f6351f3a98440": {
                "Name": "receiver",
                "EndpointID": "4088899ba7d6d6c4c706b7ba642726aaa1139ed3ca7335b26c86cdf92a85a1dd",
                "MacAddress": "82:47:d7:a2:7c:dc",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]
anudalal25@anusha:~/sig-lab$
```

Screenshot 6: Step C7: ping by name from sender to receiver, STUDENT_UID inside the container, liboqs mechanisms and container IPs



3. Key generation, signing, transfer and verification (Parts D-G)

Screenshot 7: Step D1: exact ML-DSA / SLH-DSA names in liboqs (ML-DSA-44, SLH_DSA_PURE_SHA2_128S)

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# python3 -c "import oqs; print([m for m in oqs.get_enabled_sig_mechanisms() if 'ML-DSA' in m or 'SL
H' in m])"
liboqs-python faulthandler is disabled
['ML-DSA-44', 'ML-DSA-65', 'ML-DSA-87', 'SLH_DSA_PURE_SHA2_128S', 'SLH_DSA_PURE_SHA2_128F', 'SLH_DSA_PURE_SHA2_192S'
, 'SLH_DSA_PURE_SHA2_192F', 'SLH_DSA_PURE_SHA2_256S', 'SLH_DSA_PURE_SHA2_256F', 'SLH_DSA_PURE_SHAKE_128S', 'SLH_DSA_
PURE_SHAKE_128F', 'SLH_DSA_PURE_SHAKE_192S', 'SLH_DSA_PURE_SHAKE_192F', 'SLH_DSA_PURE_SHAKE_256S', 'SLH_DSA_PURE_SHA
KE_256F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_224_PREHASH_SHA2
_192S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_224_PREHASH_SHA2_2
56F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_
192S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_224_PREHASH_SHAKE
_256F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_1
92S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_256_PREHASH_SHA2_256
F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_19
2S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_256_PREHASH_SHAKE_2
56F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_192
S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_384_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_384_PREHASH_SHA2_256F'
, 'SLH_DSA_SHA2_384_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_192S
', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_384_PREHASH_SHAKE_256
F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_192S'
, 'SLH_DSA_SHA2_512_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_PREHASH_SHA2_256F',
'SLH_DSA_SHA2_512_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_192S',
 'SLH_DSA_SHA2_512_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_PREHASH_SHAKE_256F'
, 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA2_512_224_PREHASH_
SHA2_192S', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_512_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_22
4_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_128S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_128F', 'SLH_DSA
_SHA2_512_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_192F', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_25
6S', 'SLH_DSA_SHA2_512_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_128S', 'SLH_DSA_SHA2_512_256_PREH
ASH_SHA2_128F', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_192F', 'SLH_DSA_SHA2_51
2_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA2_512_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_128S', 'SLH_
DSA_SHA2_512_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE
_192F', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA2_512_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_224_PREH
ASH_SHA2_128S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_224_PREHAS
H_SHA2_192F', 'SLH_DSA_SHA3_224_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_224_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_224_PREHASH_
SHAKE_128S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_224_PREHASH
_SHAKE_192F', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_224_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_256_PREHAS
H_SHA2_128S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_256_PREHASH_
SHA2_192F', 'SLH_DSA_SHA3_256_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_256_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_256_PREHASH_SH
AKE_128S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_256_PREHASH_S
HAKE_192F', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_256_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_384_PREHASH_
SHA2_128S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_384_PREHASH_SH
A2_192F', 'SLH_DSA_SHA3_384_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_384_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_384_PREHASH_SHAK
E_128S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_384_PREHASH_SHA
KE_192F', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_384_PREHASH_SHAKE_256F', 'SLH_DSA_SHA3_512_PREHASH_SH
A2_128S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_128F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_192S', 'SLH_DSA_SHA3_512_PREHASH_SHA2
_192F', 'SLH_DSA_SHA3_512_PREHASH_SHA2_256S', 'SLH_DSA_SHA3_512_PREHASH_SHA2_256F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_
128S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_128F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_192S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE
_192F', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_256S', 'SLH_DSA_SHA3_512_PREHASH_SHAKE_256F', 'SLH_DSA_SHAKE_128_PREHASH_SHA
2_128S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_128F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_192S', 'SLH_DSA_SHAKE_128_PREHASH_SH
A2_192F', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_256S', 'SLH_DSA_SHAKE_128_PREHASH_SHA2_256F', 'SLH_DSA_SHAKE_128_PREHASH_S
HAKE_128S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_128F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_192S', 'SLH_DSA_SHAKE_128_PREHA
SH_SHAKE_192F', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_256S', 'SLH_DSA_SHAKE_128_PREHASH_SHAKE_256F', 'SLH_DSA_SHAKE_256_P
REHASH_SHA2_128S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_128F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_192S', 'SLH_DSA_SHAKE_256_
PREHASH_SHA2_192F', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_256S', 'SLH_DSA_SHAKE_256_PREHASH_SHA2_256F', 'SLH_DSA_SHAKE_256
_PREHASH_SHAKE_128S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_128F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_192S', 'SLH_DSA_SHAKE
_256_PREHASH_SHAKE_192F', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_256S', 'SLH_DSA_SHAKE_256_PREHASH_SHAKE_256F']
root@sender:/lab#
```

Screenshot 8: Step D3: keygen.py generates the ECDSA, RSA-PSS, ML-DSA and SLH-DSA key pairs and prints their sizes

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# mkdir -p keys outbox
root@sender:/lab# python3 keygen.py ecdsa
liboqs-python faulthandler is disabled
[keygen] algorithm      : ecdsa (ECDSA)
[keygen] private key    : keys/ecdsa_private.key          138 bytes
[keygen] public key     : outbox/ecdsa_public.key          91 bytes
[keygen] generation time: 0.803 ms
root@sender:/lab# python3 keygen.py rsapss
liboqs-python faulthandler is disabled
[keygen] algorithm      : rsapss (RSA-PSS)
[keygen] private key    : keys/rsapss_private.key        1792 bytes
[keygen] public key     : outbox/rsapss_public.key        422 bytes
[keygen] generation time: 212.871 ms
root@sender:/lab# python3 keygen.py mldsa
liboqs-python faulthandler is disabled
[keygen] algorithm      : mldsa (ML-DSA-44)
[keygen] private key    : keys/mldsa_private.key         2560 bytes
[keygen] public key     : outbox/mldsa_public.key        1312 bytes
[keygen] generation time: 3.811 ms
root@sender:/lab# python3 keygen.py slhdsa
liboqs-python faulthandler is disabled
[keygen] algorithm      : slhdsa (SLH_DSA_PURE_SHA2_128S)
[keygen] private key    : keys/slhdsa_private.key          64 bytes
[keygen] public key     : outbox/slhdsa_public.key         32 bytes
[keygen] generation time: 96.388 ms
root@sender:/lab# ls -l keys outbox
keys:
total 16
-rw-rw-rw- 1 root root  138 Oct 16 14:17 ecdsa_private.key
-rw-rw-rw- 1 root root 2560 Oct 16 14:17 mldsa_private.key
-rw-rw-rw- 1 root root 1792 Oct 16 14:17 rsapss_private.key
-rw-rw-rw- 1 root root   64 Oct 16 14:17 slhdsa_private.key

outbox:
total 16
-rw-rw-rw- 1 root root   91 Oct 16 14:17 ecdsa_public.key
-rw-rw-rw- 1 root root 1312 Oct 16 14:17 mldsa_public.key
-rw-rw-rw- 1 root root  422 Oct 16 14:17 rsapss_public.key
-rw-rw-rw- 1 root root   32 Oct 16 14:17 slhdsa_public.key
root@sender:/lab#
```

Screenshot 9: Part E: sign.py creates the UID document and signs it with all four algorithms; document SHA-256 noted

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# python3 sign.py ecdsa
liboqs-python faulthandler is disabled
[sign] created outbox/document.txt
[sign] algorithm     : ecdsa
[sign] document      : outbox/document.txt (59 bytes)
[sign] signature     : outbox/ecdsa.sig (71 bytes)
[sign] signing time  : 11.142 ms
root@sender:/lab# python3 sign.py rsapss
liboqs-python faulthandler is disabled
[sign] algorithm     : rsapss
[sign] document      : outbox/document.txt (59 bytes)
[sign] signature     : outbox/rsapss.sig (384 bytes)
[sign] signing time  : 3.724 ms
root@sender:/lab# python3 sign.py mldsa
liboqs-python faulthandler is disabled
[sign] algorithm     : mldsa
[sign] document      : outbox/document.txt (59 bytes)
[sign] signature     : outbox/mldsa.sig (2420 bytes)
[sign] signing time  : 1.080 ms
root@sender:/lab# python3 sign.py slhdsa
liboqs-python faulthandler is disabled
[sign] algorithm     : slhdsa
[sign] document      : outbox/document.txt (59 bytes)
[sign] signature     : outbox/slhdsa.sig (7856 bytes)
[sign] signing time  : 777.197 ms
root@sender:/lab# cat outbox/document.txt
Transfer Rs.3032 to Account 0032
Issued by UID: 2024300032
root@sender:/lab# ls -l outbox
total 40
-rw-rw-rw- 1 root root   59 Oct 16 14:17 document.txt
-rw-rw-rw- 1 root root   71 Oct 16 14:17 ecdsa.sig
-rw-rw-rw- 1 root root   91 Oct 16 14:17 ecdsa_public.key
-rw-rw-rw- 1 root root 2420 Oct 16 14:17 mldsa.sig
-rw-rw-rw- 1 root root 1312 Oct 16 14:17 mldsa_public.key
-rw-rw-rw- 1 root root  384 Oct 16 14:17 rsapss.sig
-rw-rw-rw- 1 root root  422 Oct 16 14:17 rsapss_public.key
-rw-rw-rw- 1 root root 7856 Oct 16 14:17 slhdsa.sig
-rw-rw-rw- 1 root root   32 Oct 16 14:17 slhdsa_public.key
root@sender:/lab# sha256sum outbox/document.txt
d44e70dfc1a5a1f16b40f91fc9c652fc2a9e60d84dbf74435a3f4deb7cf7f388  outbox/document.txt
root@sender:/lab#
```

Screenshot 10: Part F: receiver downloads document, signatures and public keys from http://sender:8000; SHA-256 matches

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# mkdir -p /lab/inbox && cd /lab/inbox && curl -O http://sender:8000/document.txt
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100    59  100    59    0     0   5880      0 --:--:-- --:--:-- --:--:--  6555
root@receiver:/lab/inbox# for a in ecdsa rsapss mldsa slhdsa; do curl -sO http://sender:8000/${a}.sig; curl -sO http
://sender:8000/${a}_public.key; done
root@receiver:/lab/inbox# ls -l /lab/inbox
total 40
-rw-rw-rw- 1 root root   59 Oct 16 14:17 document.txt
-rw-rw-rw- 1 root root   71 Oct 16 14:17 ecdsa.sig
-rw-rw-rw- 1 root root   91 Oct 16 14:17 ecdsa_public.key
-rw-rw-rw- 1 root root 2420 Oct 16 14:17 mldsa.sig
-rw-rw-rw- 1 root root 1312 Oct 16 14:17 mldsa_public.key
-rw-rw-rw- 1 root root  384 Oct 16 14:17 rsapss.sig
-rw-rw-rw- 1 root root  422 Oct 16 14:17 rsapss_public.key
-rw-rw-rw- 1 root root 7856 Oct 16 14:17 slhdsa.sig
-rw-rw-rw- 1 root root   32 Oct 16 14:17 slhdsa_public.key
root@receiver:/lab/inbox# sha256sum /lab/inbox/document.txt
d44e70dfc1a5a1f16b40f91fc9c652fc2a9e60d84dbf74435a3f4deb7cf7f388  /lab/inbox/document.txt
root@receiver:/lab/inbox#
```

Screenshot 11: Part F: sender's HTTP server log showing every request coming from the receiver's IP (172.18.0.3)

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# cd /lab/outbox
root@sender:/lab/outbox# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /document.txt HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /ecdsa.sig HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /ecdsa_public.key HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /rsapss.sig HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /rsapss_public.key HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /mldsa.sig HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /mldsa_public.key HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /slhdsa.sig HTTP/1.1" 200 -
172.18.0.3 - - [16/Oct/2026 14:17:39] "GET /slhdsa_public.key HTTP/1.1" 200 -
```

Screenshot 12: Part G: verify.py on the receiver prints PASS for all four algorithms

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# python3 verify.py ecdsa
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 4.340 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py rsapss
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 1.167 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py mldsa
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.221 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py slhdsa
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.758 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab#
```

4. Sizes, tampering detection and timings (Parts G-I)

Table 2: Key, signature and transmission sizes (bytes)

Algorithm	Public key	Private key	Signature	Document	Transmitted (doc + sig + public key)
ECDSA	91	138	71	59	221
RSA-PSS	422	1792	384	59	865
ML-DSA	1312	2560	2420	59	3791
SLH-DSA	32	64	7856	59	7947

Sizes of the files actually written and transferred, measured with stat -c %s. Key formats as prescribed in Step D3: ECDSA and RSA-PSS private keys PKCS#8 DER, public keys SubjectPublicKeyInfo DER; ML-DSA and SLH-DSA raw keys from liboqs (these match the FIPS 204/205 sizes 1312/2560/2420 and 32/64/7856 B).

Screenshot 13: Part G: key, signature and document sizes (stat -c %s) on the sender and bytes received for Table 2

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# for a in ecdsa rsapss mldsa slhdsa; do echo "$a public=$(stat -c %s outbox/${a}_public.key) privat
e=$(stat -c %s keys/${a}_private.key) sig=$(stat -c %s outbox/${a}.sig) doc=$(stat -c %s outbox/document.txt)"; done
ecdsa public=91 private=138 sig=71 doc=59
rsapss public=422 private=1792 sig=384 doc=59
mldsa public=1312 private=2560 sig=2420 doc=59
slhdsa public=32 private=64 sig=7856 doc=59
root@sender:/lab# exit
exit
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# for a in ecdsa rsapss mldsa slhdsa; do echo "$a received: doc+sig+pk = $(( $(stat -c %s inbox/do
cument.txt) + $(stat -c %s inbox/${a}.sig) + $(stat -c %s inbox/${a}_public.key) )) bytes"; done
ecdsa received: doc+sig+pk = 221 bytes
rsapss received: doc+sig+pk = 865 bytes
mldsa received: doc+sig+pk = 3791 bytes
slhdsa received: doc+sig+pk = 7947 bytes
root@receiver:/lab#
```

Table 3: Verification results

Test case	ECDSA	RSA-PSS	ML-DSA	SLH-DSA	Expected
1. Original document + original signature	PASS	PASS	PASS	PASS	PASS
2. One character of document changed	FAIL	FAIL	FAIL	FAIL	FAIL
3. One bit of signature changed	FAIL	FAIL	FAIL	FAIL	FAIL
4. Correct document + signature, wrong public key	FAIL	FAIL	FAIL	FAIL	FAIL

Document	Text (tamper index = UID mod 59 = 2: 'a' (0x61) changed to '`' (0x60))
Original	Transfer Rs.3032 to Account 0032 / Issued by UID: 2024300032
Modified	Tr`nsfer Rs.3032 to Account 0032 / Issued by UID: 2024300032

Test 3 flipped bit 0 of the signature byte at index UID mod length: ECDSA byte 4 (0x00 -> 0x01), RSA-PSS byte 256 (0x9b -> 0x9a), ML-DSA byte 1492 (0x09 -> 0x08), SLH-DSA byte 5232 (0x53 -> 0x52). Test 4: the receiver generated its own key pairs; for ECDSA an independent key from the secure random generator was used (keygen.py ecdsa --independent), because a UID-derived key would be identical to the sender's key. cmp confirmed all four receiver public keys differ from the sender's.

Screenshot 14: Part H, Tests 1-2: one character of the document changed; original passes, modified document FAILS

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# python3 tamper.py doc inbox/document.txt
[tamper] file      : inbox/document.txt (59 bytes)
[tamper] index     : UID mod 59 = 2
[tamper] byte      : 0x61 -> 0x60 (bit 0 flipped)
[tamper] character : 'a' -> '`'
[tamper] original text:
Transfer Rs.3032 to Account 0032
Issued by UID: 2024300032
[tamper] modified text:
Tr`nsfer Rs.3032 to Account 0032
Issued by UID: 2024300032
[tamper] written   : inbox/document.txt.tampered
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt inbox/ecdsa_public.key inbox/ecdsa.sig    # Test 1: o
riginal
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 4.987 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt inbox/rsapss_public.key inbox/rsapss.sig    # Test 1
: original
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 1.295 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt inbox/mldsa_public.key inbox/mldsa.sig    # Test 1: o
riginal
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.263 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt inbox/slhdsa_public.key inbox/slhdsa.sig    # Test 1
: original
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.872 ms
[verify] RESULT    : PASS (VALID)
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 4.088 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 1.755 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.255 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt.tampered    # Test 2: modified document
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt.tampered
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.766 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab#
```

Screenshot 15: Part H, Test 3: one bit of each signature flipped at index UID mod length; all four FAIL

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# python3 tamper.py sig inbox/ecdsa.sig
[tamper] file      : inbox/ecdsa.sig (71 bytes)
[tamper] index     : UID mod 71 = 4
[tamper] byte      : 0x00 -> 0x01 (bit 0 flipped)
[tamper] written   : inbox/ecdsa.sig.tampered
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt inbox/ecdsa_public.key inbox/ecdsa.sig.tampered    #
Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig.tampered
[verify] time      : 3.898 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 tamper.py sig inbox/rsapss.sig
[tamper] file      : inbox/rsapss.sig (384 bytes)
[tamper] index     : UID mod 384 = 256
[tamper] byte      : 0x9b -> 0x9a (bit 0 flipped)
[tamper] written   : inbox/rsapss.sig.tampered
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt inbox/rsapss_public.key inbox/rsapss.sig.tampered
 # Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: inbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig.tampered
[verify] time      : 1.162 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 tamper.py sig inbox/mldsa.sig
[tamper] file      : inbox/mldsa.sig (2420 bytes)
[tamper] index     : UID mod 2420 = 1492
[tamper] byte      : 0x09 -> 0x08 (bit 0 flipped)
[tamper] written   : inbox/mldsa.sig.tampered
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt inbox/mldsa_public.key inbox/mldsa.sig.tampered    #
Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig.tampered
[verify] time      : 0.178 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 tamper.py sig inbox/slhdsa.sig
[tamper] file      : inbox/slhdsa.sig (7856 bytes)
[tamper] index     : UID mod 7856 = 5232
[tamper] byte      : 0x53 -> 0x52 (bit 0 flipped)
[tamper] written   : inbox/slhdsa.sig.tampered
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt inbox/slhdsa_public.key inbox/slhdsa.sig.tampered
 # Test 3: modified signature
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: inbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig.tampered
[verify] time      : 0.762 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# |
```

Screenshot 16: Part H, Test 4: receiver makes its own key pairs (independent ECDSA key) - verification with the wrong public key FAILS for all four

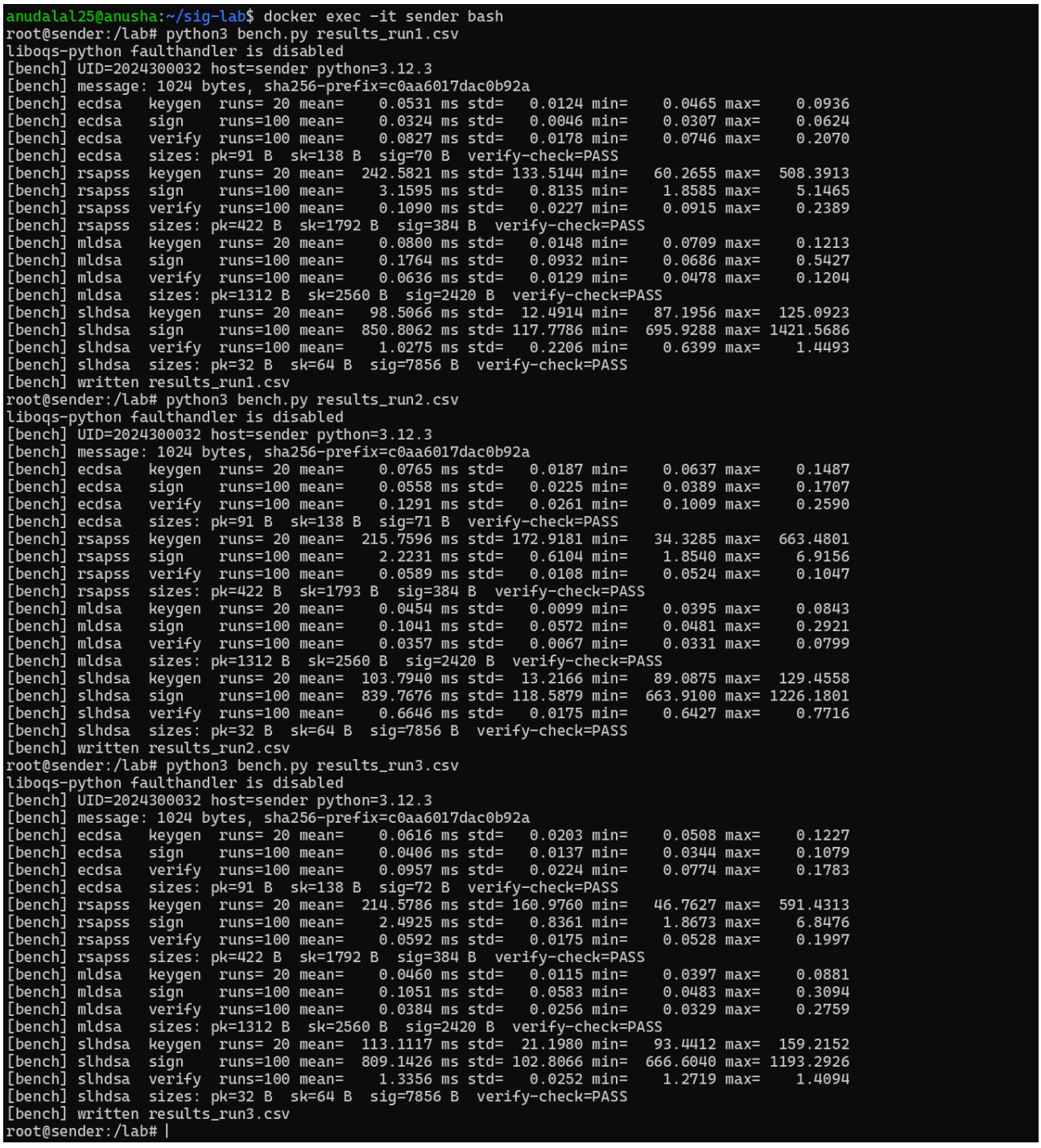
```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# mkdir -p keys outbox
root@receiver:/lab# python3 keygen.py ecdsa --independent    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : ecdsa (ECDSA)  [independent key from the secure RNG]
[keygen] private key    : keys/ecdsa_private.key          138 bytes
[keygen] public key     : outbox/ecdsa_public.key          91 bytes
[keygen] generation time: 35.439 ms
root@receiver:/lab# python3 verify.py ecdsa inbox/document.txt outbox/ecdsa_public.key inbox/ecdsa.sig    # Test 4:
wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : ecdsa
[verify] document  : inbox/document.txt
[verify] public key: outbox/ecdsa_public.key
[verify] signature : inbox/ecdsa.sig
[verify] time      : 11.863 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 keygen.py rsapss    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : rsapss (RSA-PSS)
[keygen] private key    : keys/rsapss_private.key        1794 bytes
[keygen] public key     : outbox/rsapss_public.key        422 bytes
[keygen] generation time: 31.473 ms
root@receiver:/lab# python3 verify.py rsapss inbox/document.txt outbox/rsapss_public.key inbox/rsapss.sig    # Test
4: wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : rsapss
[verify] document  : inbox/document.txt
[verify] public key: outbox/rsapss_public.key
[verify] signature : inbox/rsapss.sig
[verify] time      : 3.119 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 keygen.py mldsa    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : mldsa (ML-DSA-44)
[keygen] private key    : keys/mldsa_private.key         2560 bytes
[keygen] public key     : outbox/mldsa_public.key        1312 bytes
[keygen] generation time: 94.715 ms
root@receiver:/lab# python3 verify.py mldsa inbox/document.txt outbox/mldsa_public.key inbox/mldsa.sig    # Test 4:
wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : mldsa
[verify] document  : inbox/document.txt
[verify] public key: outbox/mldsa_public.key
[verify] signature : inbox/mldsa.sig
[verify] time      : 0.464 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# python3 keygen.py slhdsa    # Test 4: receiver makes ITS OWN key pair
liboqs-python faulthandler is disabled
[keygen] algorithm      : slhdsa (SLH_DSA_PURE_SHA2_128S)
[keygen] private key    : keys/slhdsa_private.key          64 bytes
[keygen] public key     : outbox/slhdsa_public.key         32 bytes
[keygen] generation time: 149.479 ms
root@receiver:/lab# python3 verify.py slhdsa inbox/document.txt outbox/slhdsa_public.key inbox/slhdsa.sig    # Test
4: wrong public key
liboqs-python faulthandler is disabled
[verify] algorithm : slhdsa
[verify] document  : inbox/document.txt
[verify] public key: outbox/slhdsa_public.key
[verify] signature : inbox/slhdsa.sig
[verify] time      : 0.963 ms
[verify] RESULT    : FAIL (INVALID)
root@receiver:/lab# for a in ecdsa rsapss mldsa slhdsa; do cmp -s inbox/${a}_public.key outbox/${a}_public.key && ec
ho "$a: receiver key IDENTICAL to sender key (not a wrong key!)" || echo "$a: receiver key differs from sender key";
 done
ecdsa: receiver key differs from sender key
rsapss: receiver key differs from sender key
mldsa: receiver key differs from sender key
slhdsa: receiver key differs from sender key
root@receiver:/lab#
```

Table 4: Timing results (milliseconds, mean ± standard deviation)

Algorithm	Key generation	Signing	Verification	Sign / Verify ratio
ECDSA	0.0765 ± 0.0187	0.0558 ± 0.0225	0.1291 ± 0.0261	0.43
RSA-PSS	215.7596 ± 172.9181	2.2231 ± 0.6104	0.0589 ± 0.0108	37.74
ML-DSA	0.0454 ± 0.0099	0.1041 ± 0.0572	0.0357 ± 0.0067	2.92
SLH-DSA	103.7940 ± 13.2166	839.7676 ± 118.5879	0.6646 ± 0.0175	1263.57

Sender container, median of three bench.py runs (results.csv = results_run2.csv); 1024-byte UID-seeded message; 5 warm-up calls, then key generation x20, signing x100, verification x100, each call timed with time.perf_counter_ns().

Screenshot 17: Part I: bench.py on the sender, run three times (results_run1/2/3.csv)



```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# python3 bench.py results_run1.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0531 ms std=   0.0124 min=    0.0465 max=    0.0936
[bench] ecdsa   sign    runs=100 mean=    0.0324 ms std=   0.0046 min=    0.0307 max=    0.0624
[bench] ecdsa   verify  runs=100 mean=    0.0827 ms std=   0.0178 min=    0.0746 max=    0.2070
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=70 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  242.5821 ms std= 133.5144 min=   60.2655 max=  508.3913
[bench] rsapss  sign    runs=100 mean=    3.1595 ms std=   0.8135 min=    1.8585 max=    5.1465
[bench] rsapss  verify  runs=100 mean=    0.1090 ms std=   0.0227 min=    0.0915 max=    0.2389
[bench] rsapss  sizes: pk=422 B  sk=1792 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0800 ms std=   0.0148 min=    0.0709 max=    0.1213
[bench] mldsa   sign    runs=100 mean=    0.1764 ms std=   0.0932 min=    0.0686 max=    0.5427
[bench] mldsa   verify  runs=100 mean=    0.0636 ms std=   0.0129 min=    0.0478 max=    0.1204
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=   98.5066 ms std=  12.4914 min=   87.1956 max=  125.0923
[bench] slhdsa  sign    runs=100 mean=  850.8062 ms std= 117.7786 min=  695.9288 max= 1421.5686
[bench] slhdsa  verify  runs=100 mean=    1.0275 ms std=   0.2206 min=    0.6399 max=    1.4493
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_run1.csv
root@sender:/lab# python3 bench.py results_run2.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0765 ms std=   0.0187 min=    0.0637 max=    0.1487
[bench] ecdsa   sign    runs=100 mean=    0.0558 ms std=   0.0225 min=    0.0389 max=    0.1707
[bench] ecdsa   verify  runs=100 mean=    0.1291 ms std=   0.0261 min=    0.1009 max=    0.2590
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=71 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  215.7596 ms std= 172.9181 min=   34.3285 max=  663.4801
[bench] rsapss  sign    runs=100 mean=    2.2231 ms std=   0.6104 min=    1.8540 max=    6.9156
[bench] rsapss  verify  runs=100 mean=    0.0589 ms std=   0.0108 min=    0.0524 max=    0.1047
[bench] rsapss  sizes: pk=422 B  sk=1793 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0454 ms std=   0.0099 min=    0.0395 max=    0.0843
[bench] mldsa   sign    runs=100 mean=    0.1041 ms std=   0.0572 min=    0.0481 max=    0.2921
[bench] mldsa   verify  runs=100 mean=    0.0357 ms std=   0.0067 min=    0.0331 max=    0.0799
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  103.7940 ms std=  13.2166 min=   89.0875 max=  129.4558
[bench] slhdsa  sign    runs=100 mean=  839.7676 ms std= 118.5879 min=  663.9100 max= 1226.1801
[bench] slhdsa  verify  runs=100 mean=    0.6646 ms std=   0.0175 min=    0.6427 max=    0.7716
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_run2.csv
root@sender:/lab# python3 bench.py results_run3.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=sender python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0616 ms std=   0.0203 min=    0.0508 max=    0.1227
[bench] ecdsa   sign    runs=100 mean=    0.0406 ms std=   0.0137 min=    0.0344 max=    0.1079
[bench] ecdsa   verify  runs=100 mean=    0.0957 ms std=   0.0224 min=    0.0774 max=    0.1783
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=72 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  214.5786 ms std= 160.9760 min=   46.7627 max=  591.4313
[bench] rsapss  sign    runs=100 mean=    2.4925 ms std=   0.8361 min=    1.8673 max=    6.8476
[bench] rsapss  verify  runs=100 mean=    0.0592 ms std=   0.0175 min=    0.0528 max=    0.1997
[bench] rsapss  sizes: pk=422 B  sk=1792 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0460 ms std=   0.0115 min=    0.0397 max=    0.0881
[bench] mldsa   sign    runs=100 mean=    0.1051 ms std=   0.0583 min=    0.0483 max=    0.3094
[bench] mldsa   verify  runs=100 mean=    0.0384 ms std=   0.0256 min=    0.0329 max=    0.2759
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  113.1117 ms std=  21.1980 min=   93.4412 max=  159.2152
[bench] slhdsa  sign    runs=100 mean=  809.1426 ms std= 102.8066 min=  666.6040 max= 1193.2926
[bench] slhdsa  verify  runs=100 mean=    1.3356 ms std=   0.0252 min=    1.2719 max=    1.4094
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results_run3.csv
root@sender:/lab#
```

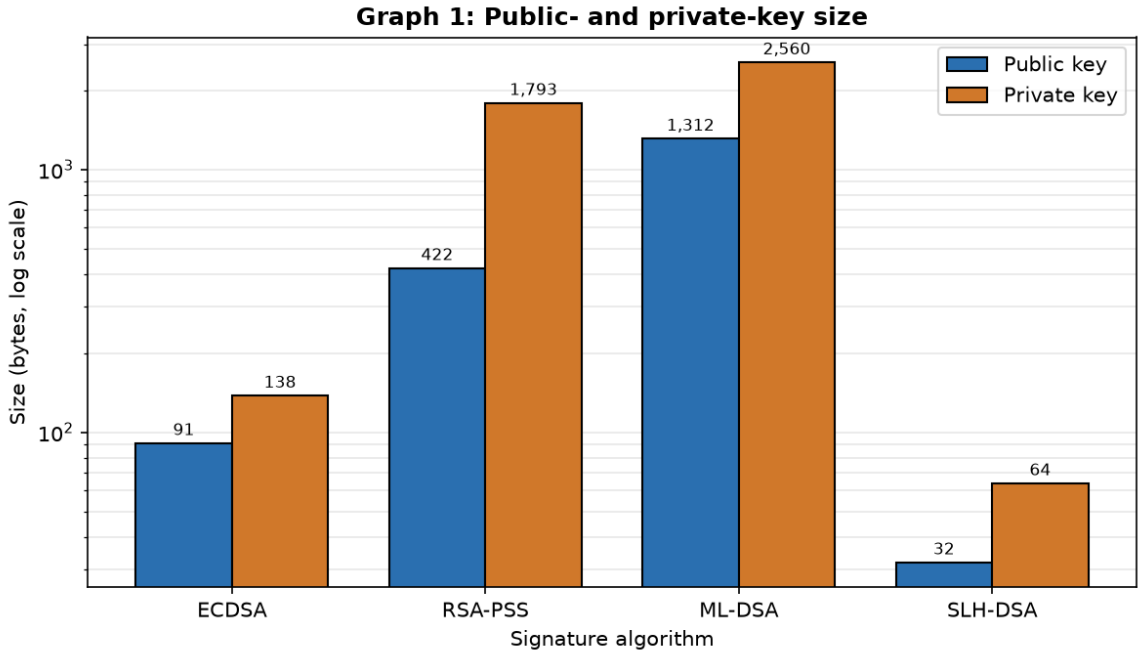
Screenshot 18: Part I: the median run (run 2) is kept as results.csv on the sender

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# for r in 1 2 3; do echo "run $r: sum of mean_ms = $(awk -F, 'NR>1{s+=$5} END{printf "%.2f", s}' re
sults_run$r.csv)"; done
run 1: sum of mean_ms = 1196.68
run 2: sum of mean_ms = 1162.71
run 3: sum of mean_ms = 1141.11
root@sender:/lab# cp results_run2.csv results.csv && cat results.csv
uid,algorithm,operation,runs,mean_ms,std_ms,min_ms,max_ms,public_key_bytes,private_key_bytes,signature_bytes
2024300032,ecdsa,keygen,20,0.0765,0.0187,0.0637,0.1487,91,138,71
2024300032,ecdsa,sign,100,0.0558,0.0225,0.0389,0.1707,91,138,71
2024300032,ecdsa,verify,100,0.1291,0.0261,0.1009,0.2590,91,138,71
2024300032,rsapss,keygen,20,215.7596,172.9181,34.3285,663.4801,422,1793,384
2024300032,rsapss,sign,100,2.2231,0.6104,1.8540,6.9156,422,1793,384
2024300032,rsapss,verify,100,0.0589,0.0108,0.0524,0.1047,422,1793,384
2024300032,mldsa,keygen,20,0.0454,0.0099,0.0395,0.0843,1312,2560,2420
2024300032,mldsa,sign,100,0.1041,0.0572,0.0481,0.2921,1312,2560,2420
2024300032,mldsa,verify,100,0.0357,0.0067,0.0331,0.0799,1312,2560,2420
2024300032,slhdsa,keygen,20,103.7940,13.2166,89.0875,129.4558,32,64,7856
2024300032,slhdsa,sign,100,839.7676,118.5879,663.9100,1226.1801,32,64,7856
2024300032,slhdsa,verify,100,0.6646,0.0175,0.6427,0.7716,32,64,7856
root@sender:/lab#
```

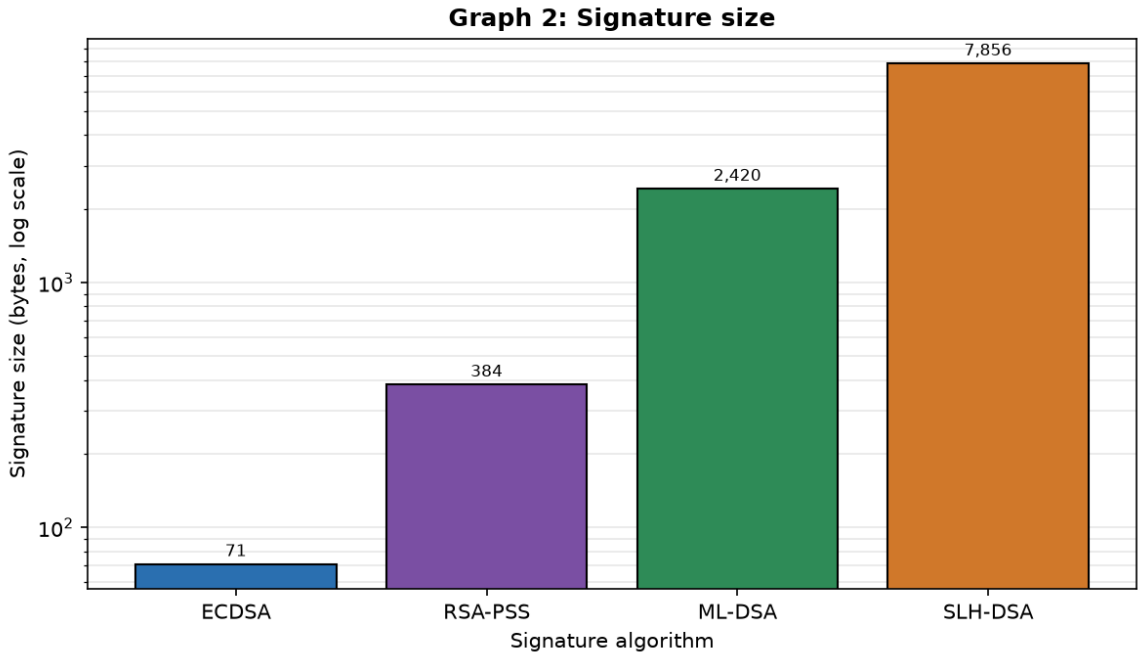
Screenshot 19: Part I: bench.py repeated on the receiver container (receiver results.csv)

```
anudalal25@anusha:~/sig-lab$ docker exec -it receiver bash
root@receiver:/lab# python3 bench.py results.csv
liboqs-python faulthandler is disabled
[bench] UID=2024300032 host=receiver python=3.12.3
[bench] message: 1024 bytes, sha256-prefix=c0aa6017dac0b92a
[bench] ecdsa   keygen  runs= 20 mean=    0.0908 ms std=   0.0204 min=    0.0787 max=    0.1586
[bench] ecdsa   sign    runs=100 mean=    0.0559 ms std=   0.0136 min=    0.0453 max=    0.1295
[bench] ecdsa   verify  runs=100 mean=    0.1522 ms std=   0.0246 min=    0.1016 max=    0.2678
[bench] ecdsa   sizes: pk=91 B  sk=138 B  sig=71 B  verify-check=PASS
[bench] rsapss  keygen  runs= 20 mean=  258.2299 ms std= 166.1175 min=   75.0116 max=  620.7739
[bench] rsapss  sign    runs=100 mean=    2.6541 ms std=   0.7561 min=    1.8775 max=    4.3344
[bench] rsapss  verify  runs=100 mean=    0.0875 ms std=   0.1650 min=    0.0527 max=    1.6344
[bench] rsapss  sizes: pk=422 B  sk=1793 B  sig=384 B  verify-check=PASS
[bench] mldsa   keygen  runs= 20 mean=    0.0471 ms std=   0.0108 min=    0.0391 max=    0.0867
[bench] mldsa   sign    runs=100 mean=    0.0954 ms std=   0.0479 min=    0.0474 max=    0.2459
[bench] mldsa   verify  runs=100 mean=    0.0365 ms std=   0.0087 min=    0.0329 max=    0.1066
[bench] mldsa   sizes: pk=1312 B  sk=2560 B  sig=2420 B  verify-check=PASS
[bench] slhdsa  keygen  runs= 20 mean=  110.9689 ms std=  17.3681 min=   91.5047 max=  150.8713
[bench] slhdsa  sign    runs=100 mean=  786.0777 ms std=  96.0449 min=  663.8267 max= 1360.1079
[bench] slhdsa  verify  runs=100 mean=    0.6878 ms std=   0.0325 min=    0.6637 max=    0.8916
[bench] slhdsa  sizes: pk=32 B  sk=64 B  sig=7856 B  verify-check=PASS
[bench] written results.csv
root@receiver:/lab#
```

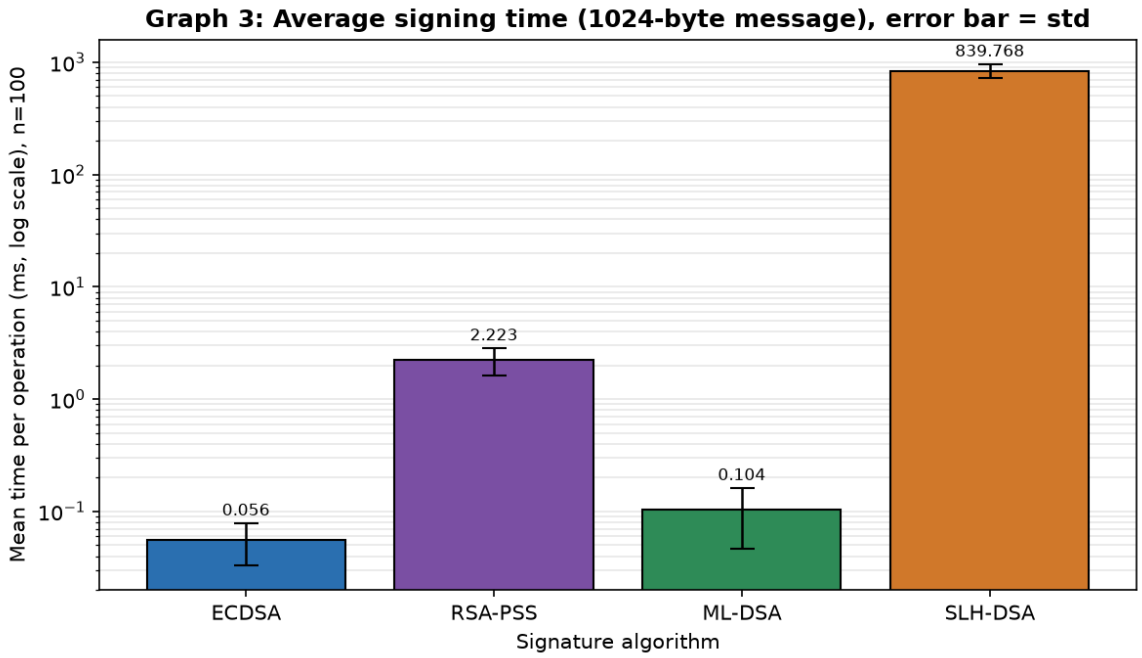
5. Graphical analysis (Section 15) and comparative analysis (Part J)



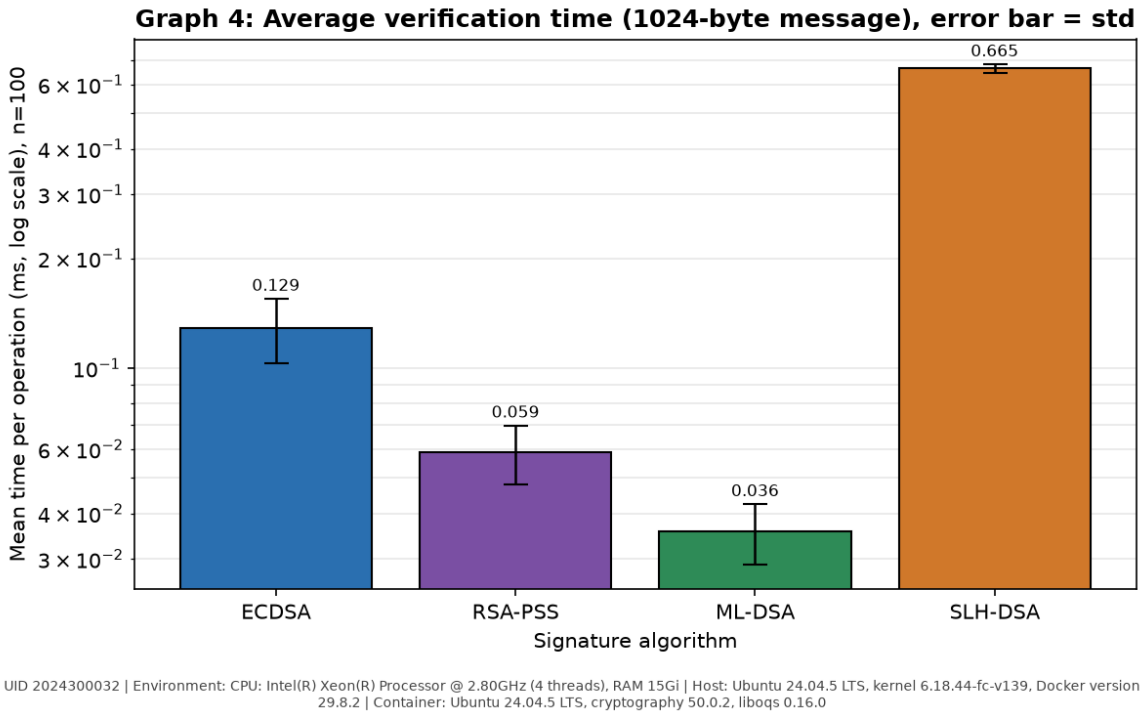
Graph 1: Public-key and private-key size (bytes, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



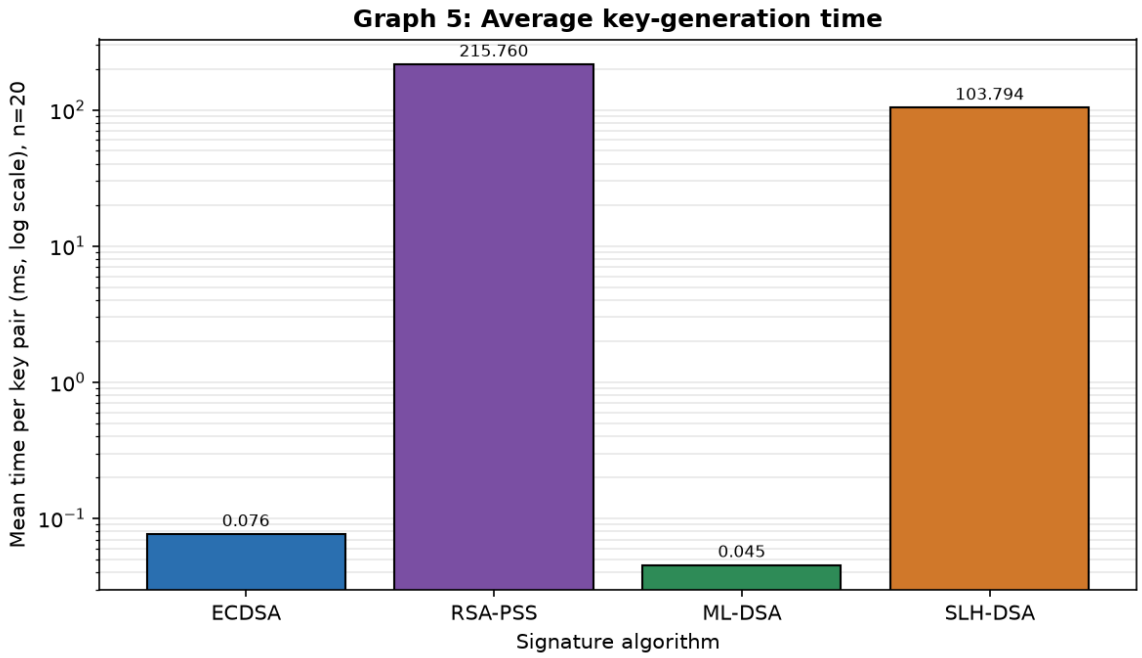
Graph 2: Signature size (bytes, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



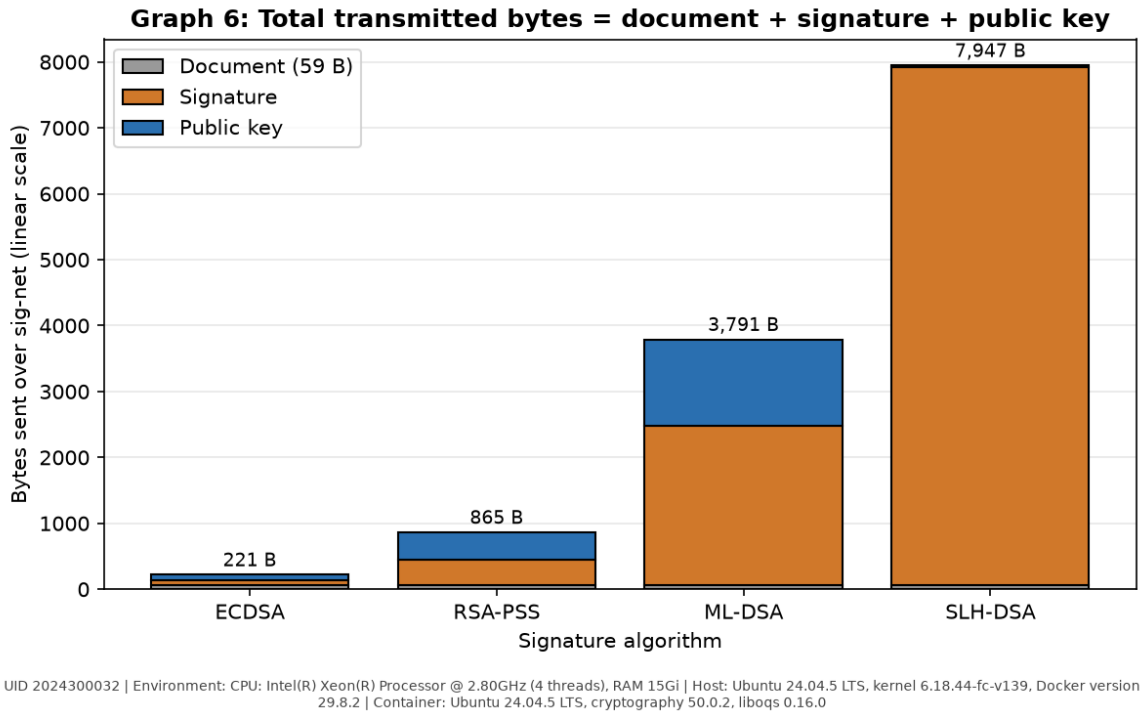
Graph 3: Average signing time (ms, log scale); error bar = standard deviation, n = 100. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



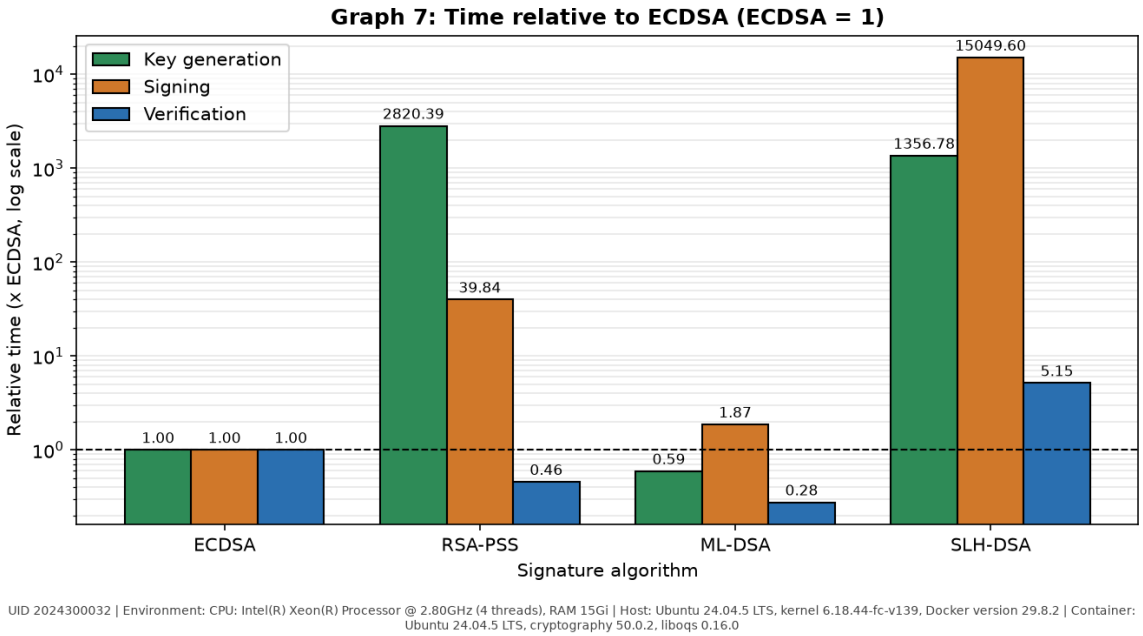
Graph 4: Average verification time (ms, log scale); error bar = standard deviation, n = 100. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 5: Average key-generation time (ms, log scale), n = 20. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 6: Total transmitted bytes = document + signature + public key. Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.



Graph 7 (extra): Sign, verify and key-generation time relative to ECDSA (ECDSA = 1, log scale). Environment: CPU: Intel(R) Xeon(R) Processor @ 2.80GHz (4 threads), RAM 15Gi; Host: Ubuntu 24.04.5 LTS, kernel 6.18.44-fc-v139, Docker version 29.8.2; Container: Ubuntu 24.04.5 LTS, cryptography 50.0.2, liboqs 0.16.0. UID 2024300032.

Screenshot 20: Section 15: plot.py draws the seven graphs; docker cp copies graphs/ to the host

```
anudalal25@anusha:~/sig-lab$ docker exec -it sender bash
root@sender:/lab# python3 plot.py results.csv outbox/document.txt
[plot] wrote graphs/graph1_key_sizes.png
[plot] wrote graphs/graph2_signature_size.png
[plot] wrote graphs/graph3_signing_time.png
[plot] wrote graphs/graph4_verification_time.png
[plot] wrote graphs/graph5_keygen_time.png
[plot] wrote graphs/graph6_transmitted_bytes.png
[plot] wrote graphs/graph7_relative_to_ecdsa.png
root@sender:/lab# exit
exit
anudalal25@anusha:~/sig-lab$ docker cp sender:/lab/graphs ./graphs
anudalal25@anusha:~/sig-lab$ ls -l graphs
total 532
-rw-rw-rw- 1 root root 70198 Oct 16 14:27 graph1_key_sizes.png
-rw-rw-rw- 1 root root 62381 Oct 16 14:27 graph2_signature_size.png
-rw-rw-rw- 1 root root 75605 Oct 16 14:27 graph3_signing_time.png
-rw-rw-rw- 1 root root 81159 Oct 16 14:27 graph4_verification_time.png
-rw-rw-rw- 1 root root 71722 Oct 16 14:27 graph5_keygen_time.png
-rw-rw-rw- 1 root root 81572 Oct 16 14:27 graph6_transmitted_bytes.png
-rw-rw-rw- 1 root root 89367 Oct 16 14:27 graph7_relative_to_ecdsa.png
anudalal25@anusha:~/sig-lab$
```

Table 5: Relative comparison with ECDSA as the baseline (measured value / ECDSA value)

Parameter	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Public key size	1.0	4.64	14.42	0.35
Signature size	1.0	5.41	34.08	110.65
Signing time	1.0	39.84	1.87	15049.60
Verification time	1.0	0.46	0.28	5.15
Key generation time	1.0	2820.39	0.59	1356.78
Total transmitted bytes	1.0	3.91	17.15	35.96

Table 6: Property comparison

Property	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Standard	FIPS 186-5	RFC 8017, FIPS 186-5	FIPS 204 (2024)	FIPS 205 (2024)
Security parameter / category	P-256 (secp256r1), ~128-bit	RSA-3072, ~128-bit	ML-DSA-44, NIST category 2	SLH-DSA-SHA2-128s, NIST category 1
Mathematical basis	Elliptic-curve discrete logarithm	Integer factorisation	Module lattices (Module-LWE / SIS)	Hash functions only (hypertree of Merkle / FORS trees)
Resistant to Shor's algorithm?	No	No	Yes	Yes
Public key size (small / medium / large)	Small (91 B)	Medium (422 B)	Large (1312 B)	Small (32 B)
Signature size	Small (71 B)	Medium (384 B)	Large (2420 B)	Very large (7856 B)
Fastest operation	Signing (0.056 ms)	Verification (0.059 ms)	Verification (0.036 ms)	Verification (0.665 ms)
Slowest operation	Verification (0.129 ms)	Key generation (215.8 ms)	Signing (0.104 ms)	Signing (839.8 ms)
Maturity / deployment	Very mature; TLS, SSH, code signing, smart cards	Very mature (decades); PKI, TLS, document signing	New (2024); being rolled out, often in hybrid mode	New (2024), conservative SPHINCS+ design; firmware, root keys
Main advantage	Smallest keys and signatures, fast signing	Very fast verification, universal support	Quantum-safe and fast (fastest verification measured)	Quantum-safe, relies only on hash functions; 32 B public key
Main disadvantage	Broken by Shor; needs secure nonces	Broken by Shor; slow key generation, larger keys	Large public key and signature	Very slow signing, very large signature

Table 7: Which algorithm would you choose?

Scenario	Chosen algorithm	Justification
Signing a firmware image for a device that stays in the field for 15 years	SLH-DSA	Must outlive quantum computers; hash-only security is the most conservative, signing is rare and verification was cheap in our tests.
Authenticating a short sensor message on a very low-bandwidth link	ECDSA	Smallest overhead (71 B signature, 91 B key) and the data is short-lived.
Signing a TLS server certificate in 2026 on a general web server	ECDSA (P-256)	Universally supported, tiny handshake; certificates live < 1 year, with a planned move to hybrid ECDSA + ML-DSA.
Root key of a certificate authority that must remain trusted for 30 years	SLH-DSA	Rarely signs, so slow signing and large signatures are acceptable; security rests only on hash functions.
High-volume server that verifies millions of signatures per second	ML-DSA	Fastest verification in our measurements (0.036 ms per signature on this setup) and quantum-safe.

Timings in Tables 4-7 are those measured in this experiment's containers on the machine in Table 1; absolute values depend on the hardware and library build, the relative ordering is what the choices rely on.

6. Answers to the discussion questions

Questions 1-11 (Section 17), handwritten.

[attach handwritten pages here]

7. Final conclusion (200-250 words)

Handwritten conclusion (Section 18).

[attach handwritten pages here]

8. Source code

Dockerfile

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 python3-pip python3-venv git cmake ninja-build build-essential \
    libssl-dev curl wget nano iproute2 iputils-ping tcpdump ca-certificates \
    && rm -rf /var/lib/apt/lists/*
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir cryptography matplotlib pandas liboqs-python
# The first import makes liboqs-python download and build liboqs (a few minutes)
RUN python3 -c "import oqs; print(oqs.oqs_version())"
WORKDIR /lab
CMD ["sleep", "infinity"]
```

common.py

```
"""common.py - shared helpers for the digital-signature lab (sender and receiver).

Contents (Step D2):
  * STUDENT_UID read from the environment (must be exactly 10 digits)
  * ALGS: lab name -> library name / parameters
  * document text, ECDSA private scalar, benchmark message, tamper index (Section 5)
  * file-path helpers and a millisecond timing helper
"""
import hashlib
import os
import random
import sys
import time

# ---------------------------------------------------------------- UID
def get_uid():
    uid = os.environ.get("STUDENT_UID", "").strip()
    if not uid:
        sys.exit("ERROR: environment variable STUDENT_UID is not set "
                 "(start the container with -e STUDENT_UID=<your 10-digit UID>)")
    if not (uid.isdigit() and len(uid) == 10):
        sys.exit(f"ERROR: STUDENT_UID must be exactly 10 digits, got {uid!r}")
    return uid


UID = get_uid()

# ---------------------------------------------------------------- algorithms
# Strings for the PQ schemes are the exact names printed by Step D1 on this system.
ALGS = {
    "ecdsa":  {"family": "classical", "label": "ECDSA",   "curve": "SECP256R1", "hash": "SHA256"},
    "rsapss": {"family": "classical", "label": "RSA-PSS", "key_size": 3072, "e": 65537,
               "hash": "SHA256", "salt_length": 32},
    "mldsa":  {"family": "pq", "label": "ML-DSA",  "oqs_name": "ML-DSA-44"},
    "slhdsa": {"family": "pq", "label": "SLH-DSA", "oqs_name": "SLH_DSA_PURE_SHA2_128S"},
}
ALG_ORDER = ["ecdsa", "rsapss", "mldsa", "slhdsa"]


def check_alg(alg):
    if alg not in ALGS:
        sys.exit(f"ERROR: unknown algorithm {alg!r}; choose one of: {', '.join(ALG_ORDER)}")
    return alg


# ---------------------------------------------------------------- UID-derived values (Section 5)
# Order n of the P-256 group
P256_N = int("FFFFFFFF00000000FFFFFFFFFFFFFFFF"
             "BCE6FAADA7179E84F3B9CAC2FC632551", 16)
```

```
def document_text(uid=UID):
    """Line 1: Transfer Rs.<A> to Account <last 4 digits of UID>, A = 1000 + (UID mod 9000)
       Line 2: Issued by UID: <UID>"""
    amount = 1000 + (int(uid) % 9000)
    return f"Transfer Rs.{amount} to Account {uid[-4:]}\nIssued by UID: {uid}\n"


def ecdsa_private_scalar(uid=UID):
    """d = ( int( SHA-256( UID || "CNS-DSIG-ECDSA-KEY" ) ) mod (n - 1) ) + 1"""
    digest = hashlib.sha256((uid + "CNS-DSIG-ECDSA-KEY").encode()).digest()
    return (int.from_bytes(digest, "big") % (P256_N - 1)) + 1


def bench_message(uid=UID):
    """1024 bytes produced by random.Random(int(UID)).randbytes(1024)"""
    return random.Random(int(uid)).randbytes(1024)


def tamper_index(length, uid=UID):
    """index = UID mod (length of document or signature in bytes)"""
    return int(uid) % length


# ---------------------------------------------------------------- file paths
KEYS_DIR, OUTBOX_DIR, INBOX_DIR = "keys", "outbox", "inbox"


def private_key_path(alg):
    return os.path.join(KEYS_DIR, f"{alg}_private.key")


def public_key_path(alg, folder=OUTBOX_DIR):
    return os.path.join(folder, f"{alg}_public.key")


def signature_path(alg, folder=OUTBOX_DIR):
    return os.path.join(folder, f"{alg}.sig")


def document_path(folder=OUTBOX_DIR):
    return os.path.join(folder, "document.txt")


def read_bytes(path):
    with open(path, "rb") as f:
        return f.read()


def write_bytes(path, data):
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    with open(path, "wb") as f:
        f.write(data)


# ---------------------------------------------------------------- timing helper
def timed_ms(func, *args, **kwargs):
    """Run func once; return (result, elapsed milliseconds) measured with perf_counter_ns."""
    t0 = time.perf_counter_ns()
    result = func(*args, **kwargs)
    t1 = time.perf_counter_ns()
    return result, (t1 - t0) / 1e6
```

keygen.py

```
"""keygen.py <alg> [--independent] - generate the key pair of the chosen algorithm (Step D3).

--independent (wrong-key test, Part H Test 4 only): the ECDSA key comes from the secure random
generator instead of the UID. Without it, the receiver would re-derive the sender's exact ECDSA
key from the same STUDENT_UID, and the "wrong" key would really be the right one.

Private key -> keys/<alg>_private.key   (never leaves the container)
Public key  -> outbox/<alg>_public.key
```

```
"""
import sys

import oqs
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.primitives.asymmetric import ec, rsa

from common import (ALGS, check_alg, ecdsa_private_scalar, private_key_path,
                    public_key_path, timed_ms, write_bytes)


def _der_pair(private_key):
    sk = private_key.private_bytes(encoding=serialization.Encoding.DER,
                                   format=serialization.PrivateFormat.PKCS8,
                                   encryption_algorithm=serialization.NoEncryption())
    pk = private_key.public_key().public_bytes(
        encoding=serialization.Encoding.DER,
        format=serialization.PublicFormat.SubjectPublicKeyInfo)
    return sk, pk


def generate(alg, independent=False):
    """Return (private_key_bytes, public_key_bytes) for alg."""
    if alg == "ecdsa" and independent:
        return _der_pair(ec.generate_private_key(ec.SECP256R1()))
    if alg == "ecdsa":
        # private key built from the UID-derived scalar d on curve P-256
        return _der_pair(ec.derive_private_key(ecdsa_private_scalar(), ec.SECP256R1()))
    if alg == "rsapss":
        p = ALGS["rsapss"]
        return _der_pair(rsa.generate_private_key(public_exponent=p["e"], key_size=p["key_size"]))
    # ML-DSA / SLH-DSA: raw public key and raw secret key from liboqs
    with oqs.Signature(ALGS[alg]["oqs_name"]) as signer:
        pk = signer.generate_keypair()
        sk = signer.export_secret_key()
    return sk, pk


def main():
    if len(sys.argv) not in (2, 3) or (len(sys.argv) == 3 and sys.argv[2] != "--independent"):
        sys.exit("usage: python3 keygen.py <ecdsa|rsapss|mldsa|slhdsa> [--independent]")
    alg = check_alg(sys.argv[1])
    independent = len(sys.argv) == 3
    (sk, pk), ms = timed_ms(generate, alg, independent)
    write_bytes(private_key_path(alg), sk)
    write_bytes(public_key_path(alg), pk)
    name = ALGS[alg].get("oqs_name", ALGS[alg]["label"])
    print(f"[keygen] algorithm      : {alg} ({name})"
          + ("  [independent key from the secure RNG]" if independent and alg == "ecdsa" else ""))
    print(f"[keygen] private key    : {private_key_path(alg):28s} {len(sk):6d} bytes")
    print(f"[keygen] public key     : {public_key_path(alg):28s} {len(pk):6d} bytes")
    print(f"[keygen] generation time: {ms:.3f} ms")


if __name__ == "__main__":
    main()
```

sign.py

```
"""sign.py <alg> - sign the UID document (Part E).

(1) create outbox/document.txt from the UID if it does not exist, (2) read it as bytes,
(3) load the private key, (4) sign, (5) write outbox/<alg>.sig, (6) print the signature size.
"""
import os
import sys

import oqs
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, padding
from cryptography.hazmat.primitives.serialization import load_der_private_key

from common import (ALGS, check_alg, document_path, document_text, private_key_path,
```

```
                    read_bytes, signature_path, timed_ms, write_bytes)


def load_private(alg, sk_bytes):
    """Classical: cryptography key object. PQ: liboqs signer holding the secret key."""
    if ALGS[alg]["family"] == "classical":
        return load_der_private_key(sk_bytes, password=None)
    return oqs.Signature(ALGS[alg]["oqs_name"], sk_bytes)


def sign_data(alg, key, data):
    if alg == "ecdsa":
        # SHA-256 with ECDSA, DER-encoded signature
        return key.sign(data, ec.ECDSA(hashes.SHA256()))
    if alg == "rsapss":
        # PSS, MGF1(SHA-256), salt length = SHA-256 digest length (32), hash SHA-256
        return key.sign(data,
                        padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                                    salt_length=ALGS["rsapss"]["salt_length"]),
                        hashes.SHA256())
    # ML-DSA / SLH-DSA: sign the whole document directly (no manual hashing)
    return key.sign(data)


def main():
    if len(sys.argv) != 2:
        sys.exit("usage: python3 sign.py <ecdsa|rsapss|mldsa|slhdsa>")
    alg = check_alg(sys.argv[1])

    doc = document_path()
    if not os.path.exists(doc):
        write_bytes(doc, document_text().encode())
        print(f"[sign] created {doc}")
    data = read_bytes(doc)

    key = load_private(alg, read_bytes(private_key_path(alg)))
    sig, ms = timed_ms(sign_data, alg, key, data)
    write_bytes(signature_path(alg), sig)

    print(f"[sign] algorithm     : {alg}")
    print(f"[sign] document      : {doc} ({len(data)} bytes)")
    print(f"[sign] signature     : {signature_path(alg)} ({len(sig)} bytes)")
    print(f"[sign] signing time  : {ms:.3f} ms")


if __name__ == "__main__":
    main()
```

verify.py

```
"""verify.py <alg> [document] [public_key] [signature] - verify a signature (Part G).

Defaults: inbox/document.txt, inbox/<alg>_public.key, inbox/<alg>.sig
Uses exactly the same parameters as sign.py; prints PASS or FAIL and the verification time.
"""
import sys

import oqs
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import ec, padding
from cryptography.hazmat.primitives.serialization import load_der_public_key

from common import (ALGS, INBOX_DIR, check_alg, document_path, public_key_path,
                    read_bytes, signature_path, timed_ms)


def load_public(alg, pk_bytes):
    """Classical: cryptography public-key object. PQ: (liboqs verifier, raw public key)."""
    if ALGS[alg]["family"] == "classical":
        return load_der_public_key(pk_bytes)
    return oqs.Signature(ALGS[alg]["oqs_name"]), pk_bytes
```

```
def verify_data(alg, key, data, sig):
    """Return True if the signature is valid, False otherwise."""
    if ALGS[alg]["family"] == "classical":
        try:
            if alg == "ecdsa":
                key.verify(sig, data, ec.ECDSA(hashes.SHA256()))
            else:
                key.verify(sig, data,
                           padding.PSS(mgf=padding.MGF1(hashes.SHA256()),
                                       salt_length=ALGS["rsapss"]["salt_length"]),
                           hashes.SHA256())
            return True
        except InvalidSignature:          # cryptography raises on failure
            return False
    verifier, pk = key
    return verifier.verify(data, sig, pk)  # liboqs returns a boolean


def main():
    if not 2 <= len(sys.argv) <= 5:
        sys.exit("usage: python3 verify.py <alg> [document] [public_key] [signature]")
    alg = check_alg(sys.argv[1])
    doc = sys.argv[2] if len(sys.argv) > 2 else document_path(INBOX_DIR)
    pub = sys.argv[3] if len(sys.argv) > 3 else public_key_path(alg, INBOX_DIR)
    sig_file = sys.argv[4] if len(sys.argv) > 4 else signature_path(alg, INBOX_DIR)

    data, sig = read_bytes(doc), read_bytes(sig_file)
    try:
        key = load_public(alg, read_bytes(pub))
        ok, ms = timed_ms(verify_data, alg, key, data, sig)
    except ValueError as exc:             # e.g. malformed key / wrong key length
        ok, ms = False, 0.0
        print(f"[verify] error: {exc}")

    print(f"[verify] algorithm : {alg}")
    print(f"[verify] document  : {doc}")
    print(f"[verify] public key: {pub}")
    print(f"[verify] signature : {sig_file}")
    print(f"[verify] time      : {ms:.3f} ms")
    print(f"[verify] RESULT    : {'PASS (VALID)' if ok else 'FAIL (INVALID)'}")
    sys.exit(0 if ok else 1)


if __name__ == "__main__":
    main()
```

tamper.py

```
"""tamper.py <doc|sig> <file> - Mallory modifies one character/bit (Part H).

index = UID mod (file length); the byte at that index is XORed with 0x01 (exactly one bit,
hence exactly one character, changes). The result is saved as <file>.tampered; the original
file is never overwritten.
"""
import sys

from common import read_bytes, tamper_index, write_bytes


def main():
    if len(sys.argv) != 3 or sys.argv[1] not in ("doc", "sig"):
        sys.exit("usage: python3 tamper.py <doc|sig> <file>")
    kind, path = sys.argv[1], sys.argv[2]
    data = bytearray(read_bytes(path))
    idx = tamper_index(len(data))
    old = data[idx]
    data[idx] ^= 0x01
    out = path + ".tampered"
    write_bytes(out, bytes(data))

    print(f"[tamper] file      : {path} ({len(data)} bytes)")
    print(f"[tamper] index     : UID mod {len(data)} = {idx}")
```

```
    print(f"[tamper] byte      : 0x{old:02x} -> 0x{data[idx]:02x} (bit 0 flipped)")
    if kind == "doc":
        print(f"[tamper] character : {chr(old)!r} -> {chr(data[idx])!r}")
        print("[tamper] original text:")
        print(read_bytes(path).decode(errors="replace"), end="")
        print("[tamper] modified text:")
        print(bytes(data).decode(errors="replace"), end="")
    print(f"[tamper] written   : {out}")


if __name__ == "__main__":
    main()
```

bench.py

```
"""bench.py [output.csv] - benchmark key generation, signing and verification (Part I).

Only the cryptographic call is inside the timed region (no Docker start-up, file I/O or
network). Per algorithm and operation: 5 un-recorded warm-up runs, then key generation x20,
signing x100, verification x100, each call timed individually with time.perf_counter_ns().
The message is the 1024-byte UID-seeded message from Section 5.
"""
import csv
import platform
import socket
import statistics
import sys
import time

from common import ALG_ORDER, ALGS, UID, bench_message
from keygen import generate
from sign import load_private, sign_data
from verify import load_public, verify_data

WARMUP = 5
RUNS = {"keygen": 20, "sign": 100, "verify": 100}


def measure(func, runs):
    for _ in range(WARMUP):                      # warm-up, not recorded
        func()
    times, result = [], None
    for _ in range(runs):
        t0 = time.perf_counter_ns()
        result = func()
        t1 = time.perf_counter_ns()
        times.append((t1 - t0) / 1e6)            # ns -> ms
    return times, result


def stats(times):
    return {"mean_ms": statistics.mean(times), "std_ms": statistics.stdev(times),
            "min_ms": min(times), "max_ms": max(times)}


def main():
    out = sys.argv[1] if len(sys.argv) > 1 else "results.csv"
    msg = bench_message()
    assert len(msg) == 1024
    print(f"[bench] UID={UID} host={socket.gethostname()} python={platform.python_version()}")
    print(f"[bench] message: 1024 bytes, sha256-prefix={__import__('hashlib').sha256(msg).hexdigest()[:16]}")

    rows = []
    for alg in ALG_ORDER:
        # key generation (the key pair produced last is used for signing/verification)
        kg_times, (sk, pk) = measure(lambda: generate(alg), RUNS["keygen"])
        priv, pub = load_private(alg, sk), load_public(alg, pk)

        sg_times, sig = measure(lambda: sign_data(alg, priv, msg), RUNS["sign"])
        vf_times, ok = measure(lambda: verify_data(alg, pub, msg, sig), RUNS["verify"])

        # the signature from the last signing loop must verify, otherwise we timed failures
        if not (ok and verify_data(alg, pub, msg, sig)):
```

```
            sys.exit(f"ERROR: {alg} signature from the benchmark did not verify")

        for op, times in (("keygen", kg_times), ("sign", sg_times), ("verify", vf_times)):
            s = stats(times)
            rows.append({"uid": UID, "algorithm": alg, "operation": op, "runs": len(times),
                         **{k: f"{v:.4f}" for k, v in s.items()},
                         "public_key_bytes": len(pk), "private_key_bytes": len(sk),
                         "signature_bytes": len(sig)})
            print(f"[bench] {alg:7s} {op:7s} runs={len(times):3d} mean={s['mean_ms']:10.4f} ms "
                  f"std={s['std_ms']:9.4f} min={s['min_ms']:10.4f} max={s['max_ms']:10.4f}")
        print(f"[bench] {alg:7s} sizes: pk={len(pk)} B  sk={len(sk)} B  sig={len(sig)} B  "
              f"verify-check=PASS")

    fields = ["uid", "algorithm", "operation", "runs", "mean_ms", "std_ms", "min_ms", "max_ms",
              "public_key_bytes", "private_key_bytes", "signature_bytes"]
    with open(out, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields)
        w.writeheader()
        w.writerows(rows)
    print(f"[bench] written {out}")


if __name__ == "__main__":
    main()
```

plot.py

```
"""plot.py [results.csv] [document.txt] - draw the seven graphs of Section 15 into graphs/.

Every graph has a title, axis labels with units, a legend where required and the execution
environment in the caption (read from environment.txt if present). A logarithmic y-axis is
used wherever the values differ by orders of magnitude.
"""
import os
import platform
import sys

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import ALG_ORDER, ALGS, UID

CSV = sys.argv[1] if len(sys.argv) > 1 else "results.csv"
DOC = sys.argv[2] if len(sys.argv) > 2 else os.path.join("outbox", "document.txt")
OUT = "graphs"
COLORS = {"ecdsa": "#2a6fb0", "rsapss": "#7a4fa3", "mldsa": "#2e8b57", "slhdsa": "#d0782a"}


def environment_caption():
    if os.path.exists("environment.txt"):
        with open("environment.txt") as f:
            env = " | ".join(line.strip() for line in f if line.strip())
    else:
        env = f"{platform.processor() or platform.machine()} | {platform.platform()}"
    return f"UID {UID} | Environment: {env}"


CAPTION = environment_caption()

df = pd.read_csv(CSV, dtype={"uid": str})
labels = [ALGS[a]["label"] for a in ALG_ORDER]
colors = [COLORS[a] for a in ALG_ORDER]
x = np.arange(len(ALG_ORDER))


def row(alg, op):
    return df[(df.algorithm == alg) & (df.operation == op)].iloc[0]


def sizes(col):
```

```
    return [int(row(a, "sign")[col]) for a in ALG_ORDER]


pk, sk, sig = sizes("public_key_bytes"), sizes("private_key_bytes"), sizes("signature_bytes")
doc = os.path.getsize(DOC)


def finish(fig, ax, name, title, ylabel, log=False, legend=False):
    ax.set_title(title, fontsize=12, fontweight="bold")
    ax.set_xlabel("Signature algorithm")
    ax.set_ylabel(ylabel)
    ax.set_xticks(x)
    ax.set_xticklabels(labels)
    if log:
        ax.set_yscale("log")
    ax.grid(axis="y", which="both", alpha=0.3)
    ax.set_axisbelow(True)
    if legend:
        ax.legend()
    fig.text(0.5, 0.01, CAPTION, ha="center", va="bottom", fontsize=6.5, wrap=True, color="#444")
    fig.tight_layout(rect=(0, 0.06, 1, 1))
    path = os.path.join(OUT, name)
    fig.savefig(path, dpi=150)
    plt.close(fig)
    print(f"[plot] wrote {path}")


def annotate(ax, bars, fmt="{:,.0f}", tops=None):
    for i, b in enumerate(bars):
        h = b.get_height()
        y = tops[i] if tops is not None else h
        ax.annotate(fmt.format(h), (b.get_x() + b.get_width() / 2, y), ha="center",
                    va="bottom", fontsize=8, xytext=(0, 2), textcoords="offset points")


def bar_with_err(op, name, title):
    mean = [row(a, op).mean_ms for a in ALG_ORDER]
    std = [row(a, op).std_ms for a in ALG_ORDER]
    fig, ax = plt.subplots(figsize=(8, 5))
    bars = ax.bar(x, mean, yerr=std, capsize=6, color=colors, edgecolor="black",
                  error_kw={"elinewidth": 1.2})
    annotate(ax, bars, "{:.3f}", tops=[m + sd for m, sd in zip(mean, std)])
    n = int(row(ALG_ORDER[0], op).runs)
    finish(fig, ax, name, title, f"Mean time per operation (ms, log scale), n={n}", log=True)


def main():
    os.makedirs(OUT, exist_ok=True)

    # Graph 1 - grouped bar: public and private key sizes
    fig, ax = plt.subplots(figsize=(8, 5))
    w = 0.38
    b1 = ax.bar(x - w / 2, pk, w, label="Public key", color="#2a6fb0", edgecolor="black")
    b2 = ax.bar(x + w / 2, sk, w, label="Private key", color="#d0782a", edgecolor="black")
    annotate(ax, b1); annotate(ax, b2)
    finish(fig, ax, "graph1_key_sizes.png", "Graph 1: Public- and private-key size",
           "Size (bytes, log scale)", log=True, legend=True)

    # Graph 2 - bar: signature size
    fig, ax = plt.subplots(figsize=(8, 5))
    annotate(ax, ax.bar(x, sig, color=colors, edgecolor="black"))
    finish(fig, ax, "graph2_signature_size.png", "Graph 2: Signature size",
           "Signature size (bytes, log scale)", log=True)

    # Graphs 3, 4 - bars with error bars (std)
    bar_with_err("sign", "graph3_signing_time.png",
                 "Graph 3: Average signing time (1024-byte message), error bar = std")
    bar_with_err("verify", "graph4_verification_time.png",
                 "Graph 4: Average verification time (1024-byte message), error bar = std")

    # Graph 5 - bar: key-generation time
    fig, ax = plt.subplots(figsize=(8, 5))
    kg = [row(a, "keygen").mean_ms for a in ALG_ORDER]
```

```
    annotate(ax, ax.bar(x, kg, color=colors, edgecolor="black"), "{:.3f}")
    finish(fig, ax, "graph5_keygen_time.png", "Graph 5: Average key-generation time",
           f"Mean time per key pair (ms, log scale), n={int(row('ecdsa', 'keygen').runs)}", log=True)

    # Graph 6 - stacked bar: document + signature + public key
    fig, ax = plt.subplots(figsize=(8, 5))
    d = np.array([doc] * len(ALG_ORDER)); s = np.array(sig); p = np.array(pk)
    ax.bar(x, d, label=f"Document ({doc} B)", color="#999999", edgecolor="black")
    ax.bar(x, s, bottom=d, label="Signature", color="#d0782a", edgecolor="black")
    bars = ax.bar(x, p, bottom=d + s, label="Public key", color="#2a6fb0", edgecolor="black")
    for b, tot in zip(bars, d + s + p):
        ax.annotate(f"{tot:,} B", (b.get_x() + b.get_width() / 2, tot), ha="center",
                    va="bottom", fontsize=9, xytext=(0, 2), textcoords="offset points")
    finish(fig, ax, "graph6_transmitted_bytes.png",
           "Graph 6: Total transmitted bytes = document + signature + public key",
           "Bytes sent over sig-net (linear scale)", legend=True)

    # Graph 7 (extra) - grouped bar relative to ECDSA
    fig, ax = plt.subplots(figsize=(9, 5))
    w = 0.26
    for i, (op, name, c) in enumerate((("keygen", "Key generation", "#2e8b57"),
                                       ("sign", "Signing", "#d0782a"),
                                       ("verify", "Verification", "#2a6fb0"))):
        base = row("ecdsa", op).mean_ms
        rel = [row(a, op).mean_ms / base for a in ALG_ORDER]
        annotate(ax, ax.bar(x + (i - 1) * w, rel, w, label=name, color=c, edgecolor="black"),
                 "{:.2f}")
    ax.axhline(1.0, color="black", linestyle="--", linewidth=1)
    finish(fig, ax, "graph7_relative_to_ecdsa.png",
           "Graph 7: Time relative to ECDSA (ECDSA = 1)",
           "Relative time (x ECDSA, log scale)", log=True, legend=True)


if __name__ == "__main__":
    main()
```

The same programs are used in the receiver container (receiver/ folder).

9. results.csv files and UID

UID: 2024300032 (first column of every result file)

Sender results.csv (median run = results_run2.csv)

uid	algorithm	operation	runs	mean_ms	std_ms	min_ms	max_ms	pk_bytes	sk_bytes	sig_bytes
2024300032	ecdsa	keygen	20	0.0765	0.0187	0.0637	0.1487	91	138	71
2024300032	ecdsa	sign	100	0.0558	0.0225	0.0389	0.1707	91	138	71
2024300032	ecdsa	verify	100	0.1291	0.0261	0.1009	0.2590	91	138	71
2024300032	rsapss	keygen	20	215.7596	172.9181	34.3285	663.4801	422	1793	384
2024300032	rsapss	sign	100	2.2231	0.6104	1.8540	6.9156	422	1793	384
2024300032	rsapss	verify	100	0.0589	0.0108	0.0524	0.1047	422	1793	384
2024300032	mldsa	keygen	20	0.0454	0.0099	0.0395	0.0843	1312	2560	2420
2024300032	mldsa	sign	100	0.1041	0.0572	0.0481	0.2921	1312	2560	2420
2024300032	mldsa	verify	100	0.0357	0.0067	0.0331	0.0799	1312	2560	2420
2024300032	slhdsa	keygen	20	103.7940	13.2166	89.0875	129.4558	32	64	7856
2024300032	slhdsa	sign	100	839.7676	118.5879	663.9100	1226.1801	32	64	7856
2024300032	slhdsa	verify	100	0.6646	0.0175	0.6427	0.7716	32	64	7856

Receiver results.csv

uid	algorithm	operation	runs	mean_ms	std_ms	min_ms	max_ms	pk_bytes	sk_bytes	sig_bytes
2024300032	ecdsa	keygen	20	0.0908	0.0204	0.0787	0.1586	91	138	71
2024300032	ecdsa	sign	100	0.0559	0.0136	0.0453	0.1295	91	138	71
2024300032	ecdsa	verify	100	0.1522	0.0246	0.1016	0.2678	91	138	71
2024300032	rsapss	keygen	20	258.2299	166.1175	75.0116	620.7739	422	1793	384
2024300032	rsapss	sign	100	2.6541	0.7561	1.8775	4.3344	422	1793	384
2024300032	rsapss	verify	100	0.0875	0.1650	0.0527	1.6344	422	1793	384
2024300032	mldsa	keygen	20	0.0471	0.0108	0.0391	0.0867	1312	2560	2420
2024300032	mldsa	sign	100	0.0954	0.0479	0.0474	0.2459	1312	2560	2420
2024300032	mldsa	verify	100	0.0365	0.0087	0.0329	0.1066	1312	2560	2420
2024300032	slhdsa	keygen	20	110.9689	17.3681	91.5047	150.8713	32	64	7856
2024300032	slhdsa	sign	100	786.0777	96.0449	663.8267	1360.1079	32	64	7856
2024300032	slhdsa	verify	100	0.6878	0.0325	0.6637	0.8916	32	64	7856

CSV header: uid,algorithm,operation,runs,mean_ms,std_ms,min_ms,max_ms,public_key_bytes,private_key_bytes,signature_bytes.
Files: results_sender.csv, results_receiver.csv (and sender/results_run1.csv, results_run2.csv, results_run3.csv).